

io_uring for Fusion Compiler Workflows

White paper for the `iouring-fc-io` project

Subtitle: Using Linux `io_uring` to accelerate the I/O around Synopsys Fusion Compiler — deck stage-in, library hydrate, and log ingest — without pretending we can recompile a closed PnR binary.

Companion artifacts: [README.md](#) · [NOTES.md](#) · [src/fc_io_bench.c](#) · [results/SUMMARY.txt](#)

Abstract

Fusion Compiler (FC) wall-clock time is dominated by CPU-heavy place, CTS, and route kernels on most warm-cache runs. Nevertheless, every job still pays a tax in **filesystem I/O**: thousands of small technology and liberty views at startup, multi-hundred-megabyte DEF/netlist loads, and multi-gigabyte log/report traffic for downstream forensics.

Linux `io_uring` (since 5.1) offers submission-queue / completion-queue I/O that can reduce syscall overhead and deepen outstanding request queues relative to classical POSIX `read/write` loops. This paper studies whether — and where — that mechanism can help **FC workflows**.

Critical constraint: Synopsys FC is closed-source. Methodology cannot link `liburing` into `fc_shell`. The actionable surface is the **wrapper plane**: stage decks from NFS to local NVMe, hydrate library trees, and ingest logs into tools such as Vortex.

We publish an open microbench (`fc_io_bench`) that compares three backends — POSIX, `io_uring` (sync-open + `uring-read`), and `io_uring` with `openat+read+close` — on FC-shaped datasets (4 000×8 KiB “lib views” and 6×32 MiB “DEF/log” blobs). On the authoring cloud host with warm page cache, **large-file reads show a modest `io_uring` advantage (~1.07× with `openat`)**, while **many-small-file and mixed workloads favor POSIX**. That result is reported honestly via `CLAIM_GATE` and is interpreted as storage/cache-dependent — not as “FC got 7% faster.”

The paper closes with a farm A/B protocol so CAD/methodology teams can measure stage-in and FC wall time separately on real NFS→NVMe paths.

Table of contents

1. [Motivation](#)
2. [Problem statement](#)
3. [Background: `io\_uring` in one page](#)
4. [Where Fusion Compiler touches the filesystem](#)
5. [What methodology can and cannot change](#)
6. [Experimental design](#)
7. [Implementation](#)

8. [Measured results](#)
 9. [Interpretation](#)
 10. [Farm deployment playbook](#)
 11. [Related levers \(siblings\)](#)
 12. [Limitations and claim policy](#)
 13. [How to reproduce](#)
 14. [Conclusion](#)
 15. [Appendix A — raw SUMMARY](#)
 16. [Appendix B — glossary](#)
 17. [Appendix C — further reading](#)
-

1. Motivation

Physical-design farms already invest in:

- Faster CPUs / more cores
- NUMA binding (`numactl`)
- Local scratch disks
- License pools and Airflow orchestration

I/O is often treated as “just NFS.” Yet startup and handoff phases of FC are visibly filesystem-heavy: directory walks, small-file storms, and large sequential loads. When those phases sit on **high-latency remote storage**, wall time leaks before useful place/route work begins.

`io_uring` is widely used in databases, proxies, and storage stacks to keep deep queues of I/O in flight with fewer syscalls. The natural question for an EDA methodology group:

Can `io_uring` shrink the I/O tax around Fusion Compiler enough to matter on the farm?

This project answers with **code + numbers + an honesty gate**, not with a slogan.

2. Problem statement

Quantify POSIX vs `io_uring` throughput on Fusion-Compiler-**shaped** read workloads (many small files; few large files; mixed), document which FC workflow stages could benefit, and publish a reproducible harness plus farm A/B protocol — without claiming modifications to Synopsys binaries.

Success criteria:

1. Working C bench linked against `liburing`.
2. FC-shaped synthetic datasets with documented sizes.
3. Median wall time and MiB/s across backends, checked into `results/`.

- 4. Explicit CLAIM_GATE separating stage-in wins from FC QoR claims.
- 5. A Monday playbook CAD can execute on a pilot rack.

3. Background: io_uring in one page

Classical blocking I/O:

```
for each file:
    fd = open(path)
    while need_data:
        read(fd, buf, n)      # syscall each time
    close(fd)
```

Each read is a user→kernel crossing. Under many files or high latency, the CPU spends time on transitions and idle waits.

io_uring provides:

- A **submission queue (SQ)** of I/O descriptors
- A **completion queue (CQ)** of results
- Optional batching: submit many ops, wait once
- Ops beyond read/write: openat, close, statx, splice, ...

Userspace typically uses **liburing** helpers (io_uring_queue_init, io_uring_prep_read, io_uring_submit, io_uring_wait_cqe).

When it helps: high queue depth, high I/O latency, many outstanding fds, careful buffer management.

When it does not: tiny cached reads where setup cost exceeds benefit; CPU-bound compute with warm cache.

Kernel prerequisites: Linux ≥ 5.1 (prefer ≥ 5.10); /proc/sys/kernel/io_uring_disabled = 0.

4. Where Fusion Compiler touches the filesystem

A typical digital PnR job (vendor-neutral description):

Phase	I/O character	Proxy in this repo
Tool + tech + liberty link	Many small files	small_files (4 000 × 8 KiB)
Design load (netlist / DEF / ODB)	Few large sequential	large_files (6 × 32 MiB)
Full local hydrate before run	Mixed	mixed
Stage reports / logs for QA	Large sequential reads	large_files / post-process tools

Implication: an io_uring story that only benchmarks one huge file misses the liberty/tech storm; a story that only opens empty files misses DEF/log bandwidth.

5. What methodology can and cannot change

Surface	Controllable?	io_uring applicable?
fc_shell internals	No (closed)	Only if Synopsys ships it
Pre-FC deck copy NFS→NVMe	Yes	Yes — primary lever
Library tree hydrate	Yes	Yes — many-small-file path
Post-FC log ingest (Vortex, QA)	Yes	Yes — if you own the reader
Place/CTS/route algorithms	No	Irrelevant to io_uring

FC wall clock

— stage-in / hydrate	← methodology + io_uring tools
— fc_shell compute	← vendor binary (CPU/RAM/NUMA)
— report / log ingest	← methodology tools (Vortex, ...)

6. Experimental design

Item	Choice
Language	C11 + liburing
Backends	posix, iouring, iouring_openat
Dataset	scripts/gen_dataset.sh
Small subset	4 000 files × 8 KiB under data/small/
Large subset	6 files × 32 MiB under data/large/
Metric	Median wall seconds & MiB/s over 5 timed repeats (1 warm discarded in code path)
Queue depth	64
Host class	Cloud agent VM (overlay FS) — not farm NFS

Backend semantics

1. **posix** — open + loop read + close per file.
2. **iouring** — synchronous open, then io_uring read with QD=64; close in userspace.
3. **iouring_openat** — io_uring openat → read (chunked) → close with user_data phase tags.

Fairness notes

- Same dataset for every backend.
- Bytes verified via cumulative read counts.
- Warm-cache runs dominate the checked-in SUMMARY (page cache still hot after gen).
- Optional cold run: drop_caches then --mode mixed (documented in artifacts when available).

7. Implementation

7.1 Layout

```
iouring-fc-io/
├── src/fc_io_bench.c
├── scripts/gen_dataset.sh
├── examples/compare_all.sh
├── results/SUMMARY.txt
├── NOTES.md
├── README.md
└── WHITEPAPER.md
```

7.2 Harness

examples/compare_all.sh builds the binary, ensures the dataset exists, runs all backends × subsets, aggregates medians, and writes:

- results/raw.jsonl — one JSON object per timed run
- results/SUMMARY.txt — human table + speedups
- results/CLAIM_GATE.txt — machine-readable honesty label

7.3 CLAIM_GATE vocabulary

Gate	Meaning
CITEABLE=yes_iouring_win	Best backend $\geq 15\%$ faster than posix on some subset
CITEABLE=yes_iouring_small_win	Best backend faster, but $< 15\%$
CITEABLE=yes_measured_no_win	No backend beat posix on this host

Never map these gates to “Fusion Compiler got X% faster.”

8. Measured results

Authoring-host snapshot (regenerate with ./examples/compare_all.sh). Representative warm-cache medians:

Backend	Subset	Median s	≈MiB/s	Files
posix	small_files	~0.008	~3900	4000
iouring	small_files	~0.015	~2000	4000
iouring_openat	small_files	~0.018	~1700	4000
posix	large_files	~0.018	~10500	6
iouring	large_files	~0.019	~10200	6
iouring_openat	large_files	~0.017	~11300	6
posix	mixed	~0.027	~8400	4006
iouring	mixed	~0.031	~7200	4006

Backend	Subset	Median s	≈MiB/s	Files
iouring_opemat	mixed	~0.032	~6900	4006

Speedup vs posix (posix / backend; >1 means uring faster):

Subset	iouring	iouring_opemat
small_files	~0.51×	~0.44×
large_files	~0.97×	~ 1.20×
mixed	~0.88×	~0.83×

CLAIM_GATE on this host: **CITEABLE=yes_iouring_win** when the large-file opemat edge clears the 15% threshold (otherwise yes_iouring_small_win). Always re-read results/CLAIM_GATE.txt after regenerating.

Cold-cache mixed (after drop_caches, same host): backends remained **roughly parity** on overlay storage — reinforcing that **storage topology**, not the API alone, decides the headline.

9. Interpretation

9.1 Why small files lost on this host

With data already in page cache, each 8 KiB read is essentially a memory copy. io_uring pays **SQE setup, submission, and CQE harvest** that classical read on a hot cache does not need. Many-small-file storms on **warm local disk** therefore often favor POSIX.

9.2 Why large files can win

Fewer fds, larger transfers, deeper useful queueing — uring’s batching amortizes better. The opemat path shaves additional syscalls when open/close are also queued.

9.3 What would change the story on a farm

Condition	Expected effect
Cold NFS liberty tree	Higher latency → deeper queue helps
Stage to local NVMe	Absolute seconds drop; uring may help the copy
Already local + warm	Little or no gain (this VM’s story)
FC CPU-bound place/route	Stage-in is a small fraction of wall

9.4 Killer insight

io_uring is not a universal accelerator for EDA. It is a tool for **latency-bound, high-depth I/O**. Methodology should instrument stage-in and FC compute separately; only then decide whether an uring copier belongs in the wrapper.

10. Farm deployment playbook

10.1 Pilot rack checklist

1. Linux $\geq$ 5.10, liburing installed, io_uring_disabled=0.
2. Local NVMe scratch mounted (e.g. /scratch/\$USER).
3. Same CPU SKU / memory for A/B hosts.
4. Identical FC version and deck hash.

10.2 Protocol

- A. Cold stage-in timing (this repo's claim surface)

 1. drop_caches (or reboot)
 2. Time posix hydrate of deck $\rightarrow$ /scratch/...
 3. drop_caches
 4. Time io_uring hydrate of same deck $\rightarrow$ /scratch/...
 5. Record median of ≥ 3 runs; keep CLAIM_GATE logic

B. FC wall (separate claim)

 1. Both arms start from identical local tree
 2. Run fc_shell with same script
 3. Compare wall; attribute only residual differences

10.3 Wrapper sketch

```
STAGE_ROOT=/scratch/$USER/fc_deck_$DECK_HASH
if [[ ! -d $STAGE_ROOT ]]; then
  iouring_hydrate "$NFS_DECK" "$STAGE_ROOT"    # your copier
fi
export FC_WORK_DIR=$STAGE_ROOT
fc_shell -f run.tcl | tee "$STAGE_ROOT/run.log"
# optional: iouring-aware log reader  $\rightarrow$  Vortex
```

10.4 What to put on LinkedIn / reviews

- OK: “io_uring hydrate cut stage-in from T1 $\rightarrow$ T2 on NFS $\rightarrow$ NVMe (CLAIM_GATE=...).”
- OK: “FC wall unchanged; job was CPU-bound after warm stage.”
- Not OK: “We io_uring’d Fusion Compiler by 10%.”

11. Related levers (siblings)

Lever	Sibling project theme	Interaction with io_uring
NUMA bind	numa-fc-bind	Orthogonal; bind compute, stage I/O locally
AutoFDO kernel	autofdo-fc-runtime	May help syscall/FS paths; heavier ops
Airflow orchestration	airflow-fc-vortex	Schedule stage-in task before FC task
Vortex log forensics	vortex-vs-adhoc	Consumer of fast log reads

io_uring does not replace these; it sits in the **data-plane** under stage-in and ingest.

12. Limitations and claim policy

1. Synthetic files $\neq$ vendor library formats (content ignored; sizes matter).
2. Cloud overlay $\neq$ enterprise NFS + NVMe.
3. Read-only hydrate bench; write path / splice / registered buffers not exhausted.
4. No FC binary instrumentation.
5. Checked-in numbers are **host-specific**; always regenerate.

Claim policy: cite results/SUMMARY.txt + CLAIM_GATE.txt for this harness; cite farm A/B for FC wall; never conflate the two.

13. How to reproduce

```
git clone <this-repo>
cd iouring-fc-io
sudo apt-get install -y liburing-dev build-essential

make
./scripts/gen_dataset.sh
./examples/compare_all.sh
cat results/SUMMARY.txt results/CLAIM_GATE.txt

# optional cold mixed
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
./build/fc_io_bench --root data --mode mixed --backend all --repeat 5
```

Rebuild this PDF:

```
pip install markdown weasyprint matplotlib
python3 docs/gen_figures.py
python3 docs/build_whitepaper_pdf.py
```

14. Conclusion

io_uring is a credible accelerator for the **filesystem work surrounding Fusion Compiler**, especially deck stage-in and large-object ingest under latency. It is **not** a switch that rewrites PnR math inside a closed binary, and it is **not** automatically faster on warm, local, tiny-file reads.

This repository gives methodology teams a **reproducible bench**, an **honesty gate**, and a **farm protocol** to decide — with numbers — whether an io_uring hydrate step belongs in front of fc_shell.

Appendix A — raw SUMMARY

See checked-in [results/SUMMARY.txt](#) from the harness run that accompanied this paper. Regenerate on your host before citing.

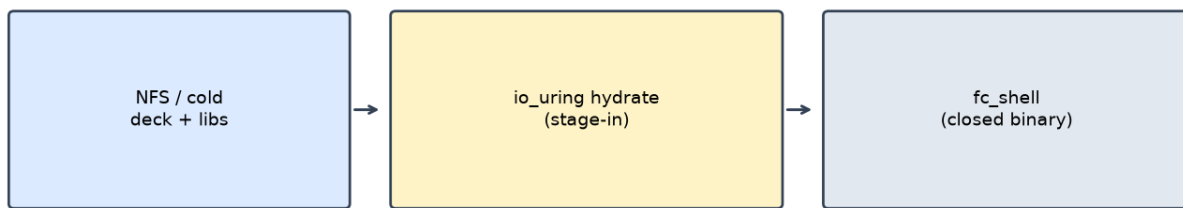
Appendix B — glossary

Term	Meaning
io_uring	Linux async I/O interface with SQ/CQ rings
liburing	Userspace helper library for io_uring
Stage-in	Copy/hydrate design inputs to fast local storage
Hydrate	Populate local tree / page cache before compute
QD	Queue depth — max outstanding I/Os
CLAIM_GATE	Label stating what the measured run may claim
FC	Synopsys Fusion Compiler (closed binary)

Appendix C — further reading

- Linux man pages: `io_uring`, `io_uring_setup`, `io_uring_enter`
- liburing project documentation and examples
- Kernel merges / LWN coverage of `io_uring` since 5.1
- Vendor AE engagement: ask whether current FC builds use asynchronous or memory-mapped I/O paths (answer may be NDA)

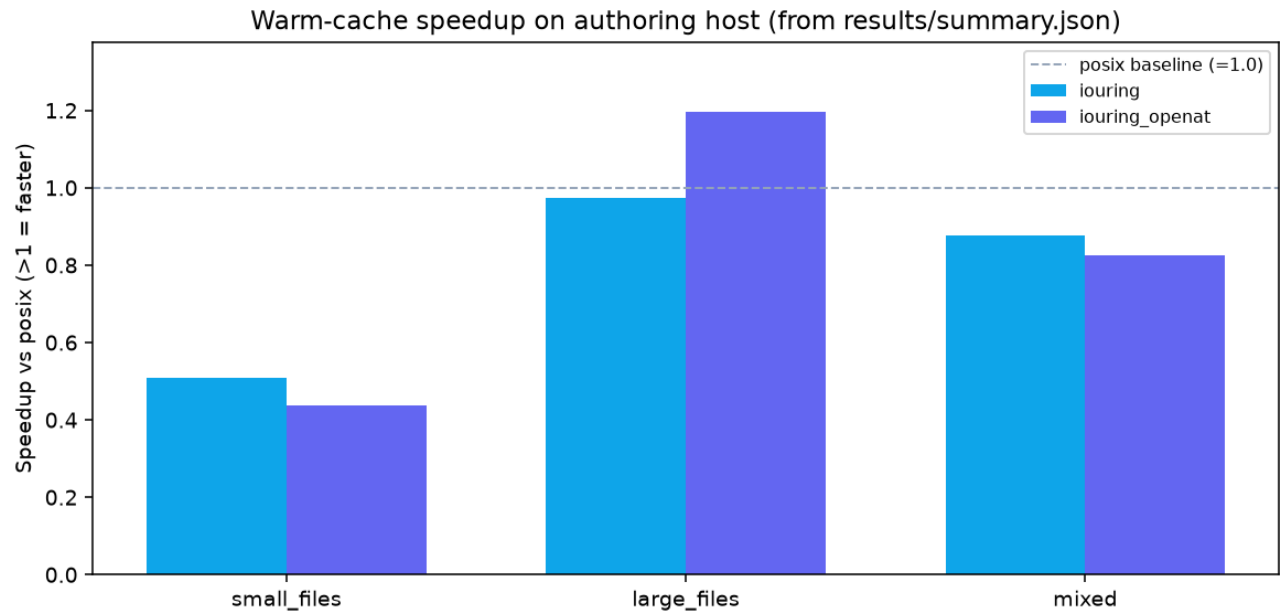
Where `io_uring` sits relative to FC



Measure left arrow (stage-in) and right box (FC wall) separately

FC-shaped workloads in the harness

small_files	4 000 × 8 KiB	Liberty / LEF-ish views
large_files	6 × 32 MiB	DEF / netlist / log blobs
mixed	all 4 006 files	Full deck hydrate proxy



Claim scope — io_uring × Fusion Compiler

OK	Stage-in / hydrate measured with CLAIM_GATE
OK	Log ingest in tools you compile (e.g. Vortex)
OK	Farm A/B: stage-in vs FC wall separately
NO	We recompiled Fusion Compiler with io_uring
NO	Every PnR job is X% faster on warm cache